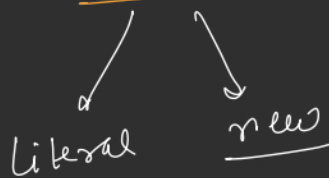


Strings



String s1 = "Hello";
String s2 = new String("Hello");

new String()
new String(String)
new String(char[])
new String(byte[])
new String(StringBuilder)
new String(StringBuffer)

String Methods

Length/Emptiness

- ↳ length()
- ↳ isEmpty()
- ↳ isBlank()

Character Access

- ↳ charAt(int)
- ↳ toCharArray()

Comparison

- ↳ equals()
- ↳ equalsIgnoreCase()
- ↳ compareTo()

Searching

- ↳ contains()
- ↳ indexOf()
- ↳ lastIndexOf()
- ↳ startsWith()
- ↳ endsWith()

Extraction/Transformation

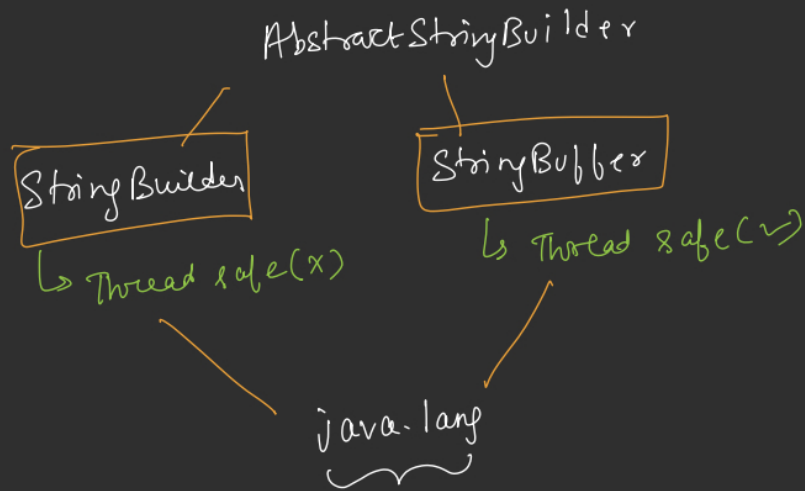
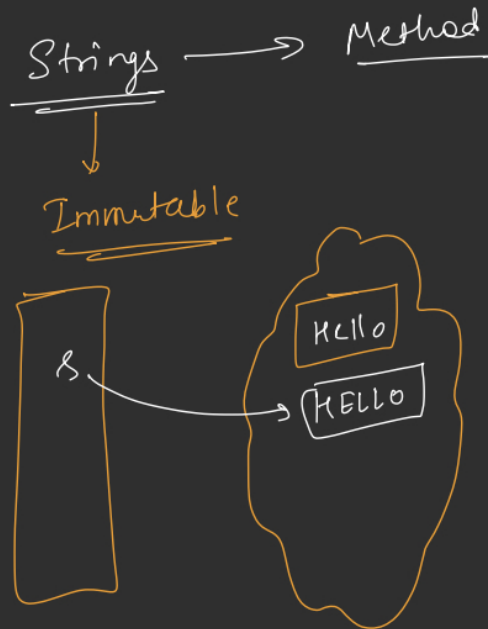
- ↳ substring()
- ↳ toUpperCase()
- ↳ toLowerCase()
- ↳ trim()
- ↳ strip()
- ↳ repeat()
- ↳ replace()
- ↳ replaceAll()
- ↳ split()
- ↳ join()

Conversion

- ↳ valueOf()
- ↳ getBytes()

Advance

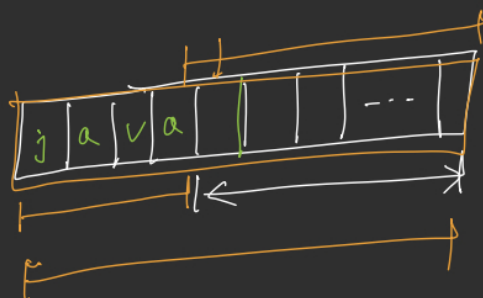
- ↳ intern()
- ↳ format()



Class String Builder extends AbstractStringBulder {
 byte[] values; // coder
 int Count;
}

String Builder sb = new String Builder ("java");

Count = 4
 Capacity = 16

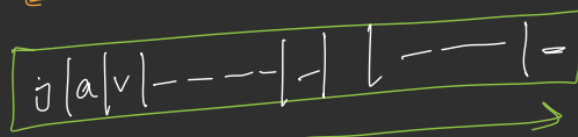
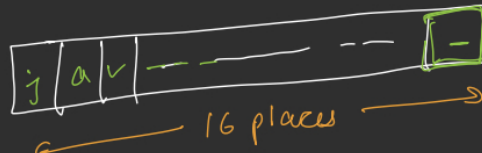


StringBuilder sb = new StringBuilder();

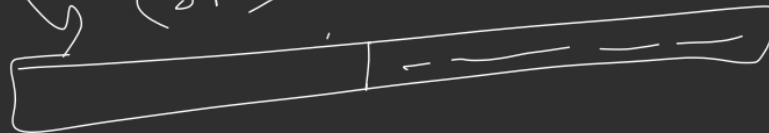
→ java

sb.append("is cool");

sb.append(" ");

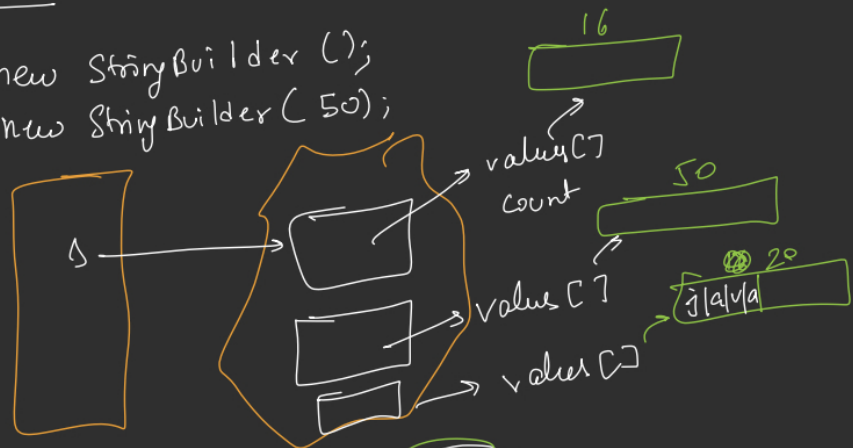


$$(34 \times 2) + 2 = 70$$



StringBuilder

sb = {
 new StringBuilder();
 new StringBuilder(50);



→ new StringBuilder("java");
 ↳ 16 + 4 = 20

String Builder / StringBuffer

- append()
- insert()
- delete()
- replace()
- reverse()
- charAt()
- setCharAt()
- length()
- capacity()
- ensureCapacity()
- trimToSize()

transformation/comparison
Strings

equals()

sb1 = new StringBuilder("Aditya");
sb2 = new StringBuilder("Aditya");

sb1.equals(sb2);
→ True X

String Builder

Mutable + thread safety

Thread 1

Thread 2

StringBuilder sb = new StringBuilder("Hello");

Thread 1:

sb.append('A');

Thread 2:

sb.append("B");

→ HelloA
→ HelloAB ✓

Race Condition

HelloB

HelloBA (X)

StringBuilder

↓
90%

→
Faster

StringBuffer

↓
10%

→ overhead